

SOLUCIONES: ENDPOINTS REST PARA CATEGORÍAS

ESTRUCTURA DE BASE DE DATOS

```
CREATE TABLE categorias (      id INT AUTO_INCREMENT PRIMARY KEY,      nombre VARCHAR(100) NOT NULL,      descripcion VARCHAR(255),      createdAt DATETIME NOT NULL DEFAULT current_timestamp(),      updatedAt DATETIME NOT NULL DEFAULT current_timestamp() );
```

```
INSERT INTO categorias (nombre, descripcion) VALUES ('Electrónica', 'Dispositivos electrónicos y gadgets'), ('Oficina', 'Material y accesorios de oficina');
```

EJERCICIO 1: POST /categorias - Registrar Nueva Categoría

```
const express = require('express'); const mysql = require('mysql2/promise');
const app = express(); app.use(express.json()); const pool =
mysql.createPool({      host: 'localhost',      user: 'root',      password:
'password',      database: 'tienda_db',      waitForConnections: true,
      connectionLimit: 10,      queueLimit: 0 }); // POST /categorias - Crear
nueva categoría app.post('/categorias', async (req, res) => {      try {
      const { nombre, descripcion } = req.body;      if
(!nombre) {      return res.status(400).json({      error:
'El nombre de la categoría es requerido'      });      }
      const conn = await pool.getConnection();      const query =
'INSERT INTO categorias (nombre, descripcion) VALUES (?, ?)';      const
[result] = await conn.execute(query, [nombre, descripcion]);
      conn.release();      res.status(201).json({
      id: result.insertId,      nombre,      descripcion,
      mensaje: 'Categoría creada exitosamente'      });      } catch
(error) {      res.status(500).json({ error: error.message });      } });
app.listen(3000, () => console.log('Servidor en puerto 3000'));
```

Nota: En Thunder Client, envíe POST a <http://localhost:3000/categorias> con body JSON: {"nombre": "Hogar", "descripcion": "Artículos para el hogar"}

EJERCICIO 2: GET /categorias - Obtener Todas las Categorías

```
// GET /categorias - Obtener todas las categorías app.get('/categorias',
async (req, res) => {    try {        const conn = await
pool.getConnection();        const query = 'SELECT * FROM categorias ORDER
BY id';        const [categorias] = await conn.execute(query);
        conn.release();        res.status(200).json({
            total: categorias.length,        categorias
        });
    } catch (error) {        res.status(500).json({ error: error.message });
    } });
```

Respuesta esperada: Lista de todas las categorías registradas en la base de datos.

EJERCICIO 3: GET /categorias/:id - Obtener Categoría con Productos

```
// Primero, asumimos que existe tabla productos con foreign key categoria_id
// CREATE TABLE productos ( // id INT AUTO_INCREMENT PRIMARY KEY, // nombre
VARCHAR(100) NOT NULL, // precio DECIMAL(10, 2), // categoria_id INT NOT
NULL, // FOREIGN KEY (categoria_id) REFERENCES categorias(id) // ); // GET
/categorias/:id - Obtener categoría y sus productos
app.get('/categorias/:id', async (req, res) => {      try {          const { id
} = req.params;          if (!id || isNaN(id)) {              return
res.status(400).json({ error: 'ID inválido' });          }
      const conn = await pool.getConnection();          // Obtener
categoría          const [categorias] = await conn.execute(
      'SELECT * FROM categorias WHERE id = ?',          [id]
    );          if (categorias.length === 0) {
      conn.release();          return res.status(404).json({ error:
'Categoría no encontrada' });          }          // Obtener productos
de la categoría          const [productos] = await conn.execute(
      'SELECT * FROM productos WHERE categoria_id = ?',
      [id]          );          conn.release();
      res.status(200).json({          categoria: categorias[0],
      productos,          totalProductos: productos.length
    });    } catch (error) {      res.status(500).json({ error:
error.message });    } });
```

EJERCICIO 4: PATCH /categorias/:id - Actualizar Categoría

```
// PATCH /categorias/:id - Actualizar categoría app.patch('/categorias/:id',
async (req, res) => {    try {        const { id } = req.params;
        const { nombre, descripcion } = req.body;        if (!id ||
isNaN(id)) {            return res.status(400).json({ error: 'ID inválido'
});        }        if (!nombre && !descripcion) {
            return res.status(400).json({            error: 'Debe
proporcionar nombre o descripción'        });        }
        const conn = await pool.getConnection();        //
Verificar que la categoría existe        const [existe] = await
conn.execute('SELECT id FROM categorias WHERE id = ?',
[id]        );        if (existe.length === 0) {
            conn.release();            return res.status(404).json({ error:
'Categoría no encontrada' });        }        // Construir query
dinámicamente        let updateFields = [];        let updateValues = [];
        if (nombre) {            updateFields.push('nombre = ?');
            updateValues.push(nombre);        }        if (descripcion) {
            updateFields.push('descripcion = ?');
            updateValues.push(descripcion);        }
        updateFields.push('updatedAt = NOW()');
        updateValues.push(id);        const query = `UPDATE
categorias SET ${updateFields.join(', ')} WHERE id = ?`;        const
[result] = await conn.execute(query, updateValues);        conn.release();
        res.status(200).json({            mensaje: 'Categoría
actualizada exitosamente',            rowsAffected: result.affectedRows
        });    } catch (error) {        res.status(500).json({ error:
error.message });    } });
```

EJERCICIO 5: DELETE /categorias/:id - Eliminar Categoría y Productos

```
// DELETE /categorias/:id - Eliminar categoría y sus productos
app.delete('/categorias/:id', async (req, res) => {    try {    const {
id } = req.params;    if (!id || isNaN(id)) {    }
    return res.status(400).json({ error: 'ID inválido' });    }
    const conn = await pool.getConnection();
    try {    // Iniciar transacción    await
conn.beginTransaction();    // Verificar que la
categoría existe    const [existe] = await conn.execute(
        'SELECT id FROM categorias WHERE id = ?',
        [id]    );    if
(existe.length === 0) {    await conn.rollback();
    conn.release();    return res.status(404).json({
error: 'Categoría no encontrada' });    }
    // Eliminar productos de la categoría    const
[productResult] = await conn.execute(        'DELETE FROM productos
WHERE categoria_id = ?',        [id]    );
    // Eliminar la categoría    const [categoryResult] =
await conn.execute(        'DELETE FROM categorias WHERE id = ?',
        [id]    );    // Confirmar
transacción    await conn.commit();    conn.release();
    res.status(200).json({    mensaje:
'Categoría y sus productos eliminados exitosamente',
        categoriasEliminadas: categoryResult.affectedRows,
        productosEliminados: productResult.affectedRows
    });    } catch (error) {    await conn.rollback();
    conn.release();    throw error;    }    } catch
(error) {    res.status(500).json({ error: error.message });    } });
```

NOTAS IMPORTANTES

1. Instalación de dependencias:

npm install express mysql2 dotenv

2. Testing con Thunder Client:

- POST: `http://localhost:3000/categorias`
Body (JSON): `{"nombre": "Tu categoría", "descripcion": "Tu descripción"}`
- GET: `http://localhost:3000/categorias`
- GET: `http://localhost:3000/categorias/1`
- PATCH: `http://localhost:3000/categorias/1`
Body (JSON): `{"nombre": "Nombre actualizado"}`
- DELETE: `http://localhost:3000/categorias/1`

3. Variables de entorno (.env):

```
DB_HOST=localhost  
DB_USER=root  
DB_PASSWORD=password  
DB_NAME=tienda_db  
PORT=3000
```

4. Manejo de errores:

- Todos los endpoints incluyen validación de entrada
- Se utiliza try-catch para manejo de excepciones
- Las transacciones en DELETE garantizan integridad de datos
- Los códigos HTTP son los estándares REST (200, 201, 400, 404, 500)